Learning Bengali Handwriting Layout from Real Pages, v2

Measurement-noise correction, fit-shrinkage correction, line clearance
and a corrected evaluation protocol for the BengaliScribe layout module

BengaliScribe / `scribe_layout_v2` working notes

12 October 2026

Abstract

BengaliScribe generates handwritten Bengali one word at a time and places the words on a page with hand-picked random constants. Version 1 of the layout module (`scribe_layout/`) replaced those constants with a page-autoregressive chain of mixture density networks (MDNs). The chain is trained on line geometry extracted from 963 real page scans (`hcr/996`). This report documents version 2 (`scribe_layout_v2/`) and is self-contained. Version 2 changes the word anchor, which is now an inverse-noise weighted mix of two height estimators. It estimates and removes anchor measurement noise from the drawn wobble, and keeps neighbouring lines at least as far apart as real lines are. It undoes the trajectory fit's shrinkage of the wobble targets, first with a line-level table and then, after an independent review found that table wrong for single words, with a per-word leverage correction. It models line curvature as noise-free by extreme deconvolution, because fitted curvature on short lines is mostly measurement noise. It also adds a learned word rotation, places words by ink extent, resets the planner at each page, keeps every line's ink inside the page margins, and corrects a biased end-to-end classifier two-sample test (C2ST). The default model is `runs/default_unfitw_dc`. A per-word correction calibrated on the actual Huber trajectory fit, rather than on least squares, was tried and is worse on all three plan-level C2STs on both test and val, so it was not adopted.

On the held-out test split (unseen texts; 5 seeds), the leverage model reaches a plan-level line C2ST AUC of 0.521 ± 0.019 and the default model, which adds the curvature deconvolution, 0.535 ± 0.012 (v1: 0.566 ± 0.021). The comparable within-page real-vs-real null is 0.496 ± 0.019, so the results are close to the null but not at it. The design choices were compared on the test split; a check on the validation split, run after the fact, gives 0.527 ± 0.030 and 0.536 ± 0.027 against v1's 0.560 ± 0.024. End-to-end (BengaliScribe render → real detector → the same extraction), page-level C2ST stays high (0.84–0.89), and the extraction still splits 11–13% of the drawn lines of the default model. We report both results and do not claim that the generated layouts are indistinguishable from real ones.

Contents

1 Summary **3**

2 Recap of v1 **4**
2.1 Data . . . 4
2.2 Geometry extraction and the writer unit U . . . 4
2.3 Trajectory fit . . . 5
2.4 The MDN chain . . . 5

3 Related work and positioning **5**

4 The v2 changes **6**
4.1 Combined word anchor and anchor-noise deconvolution . . . 6
4.2 De-noised wobble at generation time . . . 7
4.3 Line clearance by rejection sampling . . . 7

4.4 Undoing the fit shrinkage, attempt 1: a line-level table . . . 7
4.5 Undoing the fit shrinkage, attempt 2: per-word leverage . . . 8
4.6 De-noised curvature (extreme deconvolution) . . . 9
4.7 Huber-aware per-word correction (negative result, not adopted) . . . 11
4.8 Word rotation along the trajectory . . . 12
4.9 Ink-extent placement . . . 12
4.10 Planner reset per page . . . 13
4.11 Margin guard . . . 13
4.12 Balanced grouped C2ST for the end-to-end check . . . 13
4.13 Latent writer-style model (not the default; unresolved) . . . 14

5 Learned and estimated parameters **14**

6 Evaluation protocol **16**

7 Results **17**
7.1 Plan-level, 5 seeds . . . 17
7.2 Validation-split check . . . 18
7.3 FFT wobble-scale diagnostic . . . 19
7.4 2-D FFT layout-map experiment . . . 19
7.5 1-D Fourier experiment on headline traces . . . 20
7.6 End-to-end . . . 21
7.7 Qualitative BengaliScribe comparisons . . . 24

8 Corrections from independent review **25**

9 Limitations and threats to validity **27**

10 Reproduction and files **28**

1 Summary

What v2 changes. Twelve changes, each described in Section 4 as motivation → method → parameters → evidence:

1. a combined word anchor with noise deconvolution (§4.1);
2. the drawn wobble de-noised by a factor ≈ 0.83 (§4.2);
3. rejection sampling for line clearance (§4.3);
4. a line-level "unfit" table that undoes fit shrinkage (§4.4), which had a per-word bug;
5. a per-word leverage correction that fixes that bug (§4.5);
6. a curvature head that models noise-free curvature by extreme deconvolution, built on top of the leverage model. This is the default model, `runs/default_unfitw_dc` (§4.6);
7. word rotation $\beta = 0.524\times$ the line tangent angle (§4.8);
8. word placement by ink extent (§4.9);
9. a planner reset at each page (§4.10);
10. a margin guard that keeps each line's ink inside the top and bottom margins and replans a line that would cross the bottom one on a new page (§4.11);
11. a balanced grouped C2ST for the end-to-end check (§4.12);
12. a latent writer-style model, whose context bug is only partly fixed and which is not the default (§4.13).

The default model is `runs/default_unfitw_dc` (item 6). One further change was tried and rejected: a per-word correction calibrated on the Huber trajectory fit instead of least squares (`runs/default_unfitwh_dc`) is worse than the default on line, page and FFT C2ST on both test and val (§4.7).

Measured results, in brief.

- *Plan-level* (generator output re-measured by the extraction code; test split; 5 seeds): line C2ST v1 0.566 ± 0.021, v2 with the old table 0.539 ± 0.016, v2 with the leverage correction 0.521 ± 0.019, and the default model (leverage plus curvature deconvolution) 0.535 ± 0.012. Within-page null 0.496 ± 0.019. Page C2ST: 0.654, 0.660, 0.581 and 0.590 respectively (Table 8).
- *Validation split*, checked after the fact with the same protocol (§7.2): line C2ST v1 0.560 ± 0.024, old table 0.566 ± 0.028, leverage 0.527 ± 0.030, default 0.536 ± 0.027; page C2ST 0.631, 0.660, 0.575 and 0.593. The leverage-family models beat v1 on val too. The old table's test advantage over v1 on line C2ST did not replicate.
- *Controlled 2-D FFT* page-spectrum C2ST (test): v1 0.567 ± 0.034, old table 0.653 ± 0.024, leverage 0.566 ± 0.025, default 0.550 ± 0.020. The old table's regression on this metric is what exposed the per-word overshoot (§7.3).
- *Curvature* (§4.6): the share of real test lines with $n \geq 5$ whose sag exceeds $0.5\,U$ is 0.108; the leverage model gives 0.175 and the default model 0.093. The leverage model and the default model differ within seed noise on every C2ST, on test and on val. The default was chosen for its curvature and for the halved end-to-end line splitting, not for C2ST.
- *End-to-end*, balanced C2ST, model with the old table (`e2e_pagereset`): line C2ST v2 0.684, v2 with writer context 0.619, manual 0.905; page C2ST 0.834, 0.889 and 0.999. Inter-word gap W_1: 0.404, 0.267 and $0.685\,U$.

- *End-to-end with the leverage model* (`e2e_unfitw`, same protocol): line C2ST v2 0.699 and v2_ctx 0.613; page C2ST 0.851 and 0.842; gap W_1 0.376 and $0.197\,U$; pitch W_1 0.346 and $0.480\,U$, about half the old-table values (Table 12).
- *End-to-end with the default model* (`e2e_unfitw_dc`, same protocol): line C2ST v2 0.719 and v2_ctx 0.654; page C2ST 0.844 and 0.847; gap W_1 0.387 and $0.183\,U$. The extraction splits 52 of 469 drawn v2 lines (11%), against 89 of 462 (19%) with the leverage model. Slope and drift W_1 rise, but the split lines are the steep and bowed ones, so these W_1 values are not comparable across runs with different split rates (§7.6).
- *Final end-to-end result* (`e2e_final`: the default model plus the margin guard, same protocol; Table 13): line C2ST v2 0.719 and v2_ctx 0.619; page C2ST 0.850 and 0.845; gap W_1 0.425 and $0.198\,U$. The extraction splits 57 of 453 drawn v2 lines (12.6%). This is the same as `e2e_unfitw_dc` within noise.
- *Negative result* (§4.7): the Huber-aware per-word correction scores line/page/FFT C2ST 0.544/0.614/0.583 on test and 0.551/0.619/0.551 on val, against 0.535/0.590/0.550 and 0.536/0.593/0.535 for the default. It was not adopted.

How to read the numbers. Lower W_1 and lower AUC are better. An AUC of 0.5 means the classifier cannot separate the two samples *with these features and this amount of data*. It does not mean the samples are indistinguishable in general. All numbers come from the run files named in each table. Where a number comes from a comparison script whose output was not written to a run file, the table says so.

2 Recap of v1

2.1 Data

`hcr/996/Images/` holds 995 RGB scans of unruled A4 pages at 300 dpi. Each page is a handwritten copy of one of 27 printed source texts. There are no word boxes, no line boxes and no writer IDs. After excluding unusable pages (remarks such as upside-down, ruled or printed; "N/A" text; too few words) and a run of 28 typeset pages missed by the metadata (`image_744`–`771`), 963 pages remain usable. The split is *by source text*, with seed 0:

split	texts	pages	prose lines (v1 extraction)	inter-word gaps
train	18	614	5312	25 644
val	4 {1, 8, 20, 22}	144	1078	5 000
test	5 {4, 6, 11, 15, 18}	193	1488	7 032

v2 re-extracts with the combined anchor (§4.1). Its test split has 1495 lines on the same 193 pages (`runs/e2e_pagereset/results.md`).

2.2 Geometry extraction and the writer unit U

The `unified_cascade` word detector (Swin-T Mask R-CNN; configuration `p1_best`) supplies 50-point word polygons. Lines are rebuilt from the words, because the detector's own line polygons merge sloped lines. Ink is binarised with Otsu's threshold, capped at grey level 170. Each word yields its ink extents, its headline (*matra*) row with a confidence m_{frac}, its median ink row *mid*, and its core height (80th minus 20th percentile of ink rows). Missed ink is recovered as "orphans". Words are then chained greedily into lines, chain fragments are merged, words the detector merged are split at blank-column runs, and the trailing digit legend is tagged and excluded.

All geometry is expressed in a per-page **writer unit** U: the median word core height. On 300 dpi scans it ranges over 31–54 px (5th–95th percentile), about 3.5 mm. Among the three candidate units, U gave the most stable ratio of line pitch to unit across writers (CV 0.189). BengaliScribe's "body" measures $1.114\,U$.

2.3 Trajectory fit

For a line of n words with centres x_k and anchors y_k (in U), with $x_m = \frac{1}{2}(x_1 + x_n)$,

$$y_k = c_0 + c_1\,(x_k - x_m) + c_2\,(x_k - x_m)^2 + r_k, \tag{1}$$

fitted by Huber IRLS (5 iterations, $c = 1.345$). c_1 is the slope, c_2 the curvature, r_k the per-word residuals, and $s = 1.4826\,\mathrm{MAD}(r)$ the line's residual scale ("wobble"). The model order depends on n: quadratic for $n \geq 5$, linear for $n = 3, 4$, offset only for $n = 2$. Centring at x_m removes the basis collinearity between c_1 and c_2 (Spearman $-0.60 \to -0.04$). In v1 a word's anchor was its headline row when $m_{\mathrm{frac}} \geq 0.25$, and otherwise *mid* minus the page's median (*mid*−headline) offset. The same function, `dataset.measure_page`, measures both real and generated layouts.

On the training split the key statistics are: inter-word gap median $1.14\,U$, lag-1 Spearman correlation of gaps within a line 0.49; slope ICC over pages 0.77, with consecutive-line slope correlation 0.87; residual scale median $0.098\,U$, ICC 0.08; residual lag-1 -0.40, mostly an artefact of the fit.

2.4 The MDN chain

Every generator outputs, per line, a `LinePlan` $(x_{\mathrm{start}}, g_{1..n-1}, y_c, c_1, c_2, r_{1..n})$. Word k is placed at $x_k^{(0)} = x_{\mathrm{start}} + \sum_{j<k}(w_j + g_j)$, with headline row $y_c + c_1 d_k + c_2 d_k^2 + r_k$, where $d_k = x_k - x_m$. The model is autoregressive over the page:

$$\begin{aligned} p(\mathrm{page}) = \prod_{\ell} & p(c_{1,\ell} \mid H_\ell)\, p(c_{2,\ell} \mid H_\ell, c_{1,\ell})\, p(\log(s_\ell{+}0.02) \mid H_\ell, c_{1,\ell}, c_{2,\ell})\, p(x_{\mathrm{start},\ell} \mid \cdot)\, p(\mathrm{pitch}_\ell \mid \cdot) \\ & \times \prod_k p\big(g_{\ell,k} \mid w_{\ell,k}, w_{\ell,k+1}, x_{\ell,k}, g_{\ell,k-1}, \bar{g}_{\ell,<k}, c_{1,\ell}, c_{2,\ell}, s_\ell, H_\ell^{\mathrm{gap}}\big) \prod_k p\big(r_{\ell,k} \mid s_\ell, k/n_\ell\big). \end{aligned} \tag{2}$$

H_ℓ summarises the page's earlier lines: count, mean and last slope, mean curvature, mean log residual scale, mean and last pitch, mean line start, gap statistics, and missing flags. Each factor is a 1-D Gaussian mixture ($K = 4$) whose parameters come from a two-layer MLP (width 64, SiLU). The weights are fitted by maximum likelihood with teacher forcing (Adam, learning rate 10^{-3}, weight decay 10^{-5}, early stopping on validation NLL with patience 30). Targets that a line cannot provide are masked out of the likelihood. Writer-specific generation feeds a real page's measured lines in as history. A streaming `PagePlanner` (`begin_line`/`next_gap`/`end_line`) drives BengaliScribe's greedy line wrapping.

v1 shipped without test results: the result tables in its report were never filled in. The v1 numbers in this report come from the v2 evaluation harness, which re-evaluates the v1 checkpoint `scribe_layout/runs/default/model.pt`.

3 Related work and positioning

We surveyed ten papers that apply Fourier methods to handwriting or to sequence modelling (`docs/handwriting_layout_research_gap.md`).

- **Glyph-level Fourier features** for recognition: contour Fourier descriptors (Lai & Suen, 1981) and 1-D Fourier features of offline digit images (Liu et al., 2002). They describe single glyphs and say nothing about placement.
- **Learned spectral filters on online pen trajectories** for writer verification: SPECTRUM (Zhang et al., ACM MM 2025). It is discriminative, needs online x, y and pressure data, and its frequency branch helped only in multi-scale fusion.
- **Frequency-aware word-image synthesis**: FW-GAN (Khoa et al., 2025). It works on single word crops. Its authors note that FID, KID and recognition scores do not capture spacing regularity.

- **Generic spectral operators and mixers**: FNO, FNet, FNetAR and FourierFT. They need uniform grids or long sequences. With 4–8 gaps per line, a gap spectrum carries little more than a lag-1 correlation.
- **FF-ViT** (Xu et al., 2024) regresses a single global probe angle from blur in endomicroscopy images. It does not model where content sits in the image.

None of these papers learns a generative model of page layout (line trajectories, gap sequences, line spacing) from offline scans without pen data or layout annotations. None models the noise of its own geometry measurements before training a generator. None evaluates layout realism by re-measuring generated pages with the same extractor and running distribution tests. v2 contributes to the second and third points: the noise deconvolution of §4.1 and the re-measurement protocols of §6. Section 5 of the survey predicted where spectral methods would be the wrong tool: short sequences, writer habit at frequency zero, and cross-line constraints. Our 1-D and 2-D Fourier experiments (§7.5, §7.4) test those predictions.

4 The v2 changes

4.1 Combined word anchor and anchor-noise deconvolution

Motivation. The v1 anchor is the detected headline row. The v1 report already noted that headline misdetection puts some anchors on the wrong row, so part of the measured wobble is measurement noise. A generator trained on the raw residuals reproduces that detector noise as if it were handwriting.

Method. Every word has two height estimators, and a real vertical shift t of the word moves both: $A = t + e_A$ is the headline row, and $C = t + e_C$ is the median ink row minus the page's median ($mid-$headline) offset. If e_A and e_C are independent of each other and of t, the residuals r_A, r_C of the trajectory fit (1), applied to each estimator separately, satisfy

$$\operatorname{cov}(r_A, r_C) = \sigma_t^2, \qquad \operatorname{var}(r_A) - \sigma_t^2 = \sigma_A^2, \qquad \operatorname{var}(r_C) - \sigma_t^2 = \sigma_C^2. \tag{3}$$

The v2 anchor is the inverse-noise weighted combination

$$a = w\,A + (1 - w)\,C, \qquad w = \frac{\sigma_A^{-2}}{\sigma_A^{-2} + \sigma_C^{-2}}, \qquad \sigma_{\text{comb}}^2 = \left(\sigma_A^{-2} + \sigma_C^{-2}\right)^{-1}. \tag{4}$$

Words with $m_{\text{frac}} < 0.25$ use C alone (`extract.anchor`).

Parameters and how they were estimated. The estimator is `anchor_noise.py`. It uses train-split lines with at least 5 headline-confident words, 21 880 words in all, and winsorises residuals at the 1st and 99th percentiles (`runs/anchor_noise.json`). It gives $\sigma_t^2 = 0.00906\,U^2$, $\sigma_A^2 = 0.01176$, $\sigma_C^2 = 0.00631$, $w = 0.349$ and $\sigma_{\text{comb}}^2 = 0.00411$. The headline-only noise share is $\sigma_A^2 / \operatorname{var}(r_A) = 0.565$. For the combined anchor, the share is $0.00411/(0.00906 + 0.00411) = 0.31$ (computed from the same file).

Evidence and caveats. The independence assumption does not hold exactly (review M3, §8). The reviewer added two more estimators, body bottom and a longest-run headline, and the one-factor tetrad test fails: errors from the same mechanism correlate. Across the plausible values of σ_t^2, the *headline-only* noise share lies anywhere between 38% and 66%. The winsorising level also moves it: 0.61 without clipping, 0.526 at 5%. We therefore do not report "57% of the wobble is noise" as a finding. The *combined-anchor* share, which is what the generator uses, stays at 0.29–0.33 across clip levels.

4.2 De-noised wobble at generation time

Motivation. The model learns the *measured* per-word wobble, which includes anchor noise. The real handwriting should only receive the physical part. Generated pages are re-measured by the same extractor, which adds its own noise back on top.

Method. Drawn residuals are multiplied by

$$f_{\text{wobble}} = \sqrt{1 - \frac{\sigma^2_{\text{comb}}}{\sigma^2_t + \sigma^2_{\text{comb}}}} \tag{5}$$

(`bscribe_layout.wobble_factor`). Fitting shrinks noise and true wobble by the same factor under the i.i.d. assumption, so the noise share is the same at every line length. The review checked that this does not double-count.

Parameters. $f_{\text{wobble}} = \sqrt{1 - 0.31} \approx 0.83$, derived from the train-split values of §4.1. Over the combined-share range 0.29–0.33 the factor is 0.82–0.84, so the M3 uncertainty barely affects it.

Evidence. Fig. 5 (middle panel) shows the same generated words with the factor applied. The effect is subtle at page scale. There is no quantitative isolation of this step alone. The FFT diagnostic of §7.3 found ×0.83 (0.658) better than ×1.0 (0.683), but that was measured on the old-table model.

4.3 Line clearance by rejection sampling

Motivation. The model samples each line's slope and curvature from the page history without regard to the previous line's trajectory, so neighbouring lines can converge or touch. Real neighbouring lines almost never do. Without rejection, 9–15% of generated neighbouring line pairs violate the real 1st-percentile clearance, against 1.4% for real pairs (`runs/eval/ablation.md`, `clearance_violation_rate`).

Method. For consecutive lines that overlap by at least $5\,U$ in x, let $d(x)$ be the vertical distance between their trajectories, sampled at 40 points over the overlap. A new line is accepted if

$$\min_x d(x) \;\geq\; d_{\min} \quad \text{and} \quad \frac{\min_x d(x)}{\overline{d(x)}} \;\geq\; \rho_{\min}. \tag{6}$$

Otherwise the line's slope, curvature and log residual scale are redrawn (`planner.resample_shape`), up to 30 times. If no draw passes, the line is pushed down just far enough to pass.

Parameters. $d_{\min} = 2.603\,U$ and $\rho_{\min} = 0.797$ are the 1st percentiles over 3 740 consecutive real line pairs (both lines $n \geq 3$) on the train split. The val split gives 2.613 and 0.788 (`runs/line_clearance.json`). The cap of 30 redraws was set by hand.

Evidence. In the `e2e_unfitw` run log, over 20 generated documents (v2 and v2_ctx, 5 writers × 2 seeds), 179 of 941 lines (19%) needed at least one redraw, and 61 (6.5%) were pushed down after 30 failed redraws. Per document the counts were 3–19 redrawn and 0–9 pushed, out of 41–55 lines. The plan-level evaluation in Table 8 does *not* apply the rejection, which runs only inside BengaliScribe's layout. Fig. 5 (right) shows the result on the page.

4.4 Undoing the fit shrinkage, attempt 1: a line-level table

Motivation. The training targets r_k and s are residuals of the fit (1), which absorbs part of each word's offset. When a model reproduces those residuals and the evaluation re-fits the generated anchors, the wobble is shrunk *twice*. For a 5-word line, the re-measured scale drops to about 0.6 of the sampled one.

Method. By simulation (`unfit.py`; 4000 lines of i.i.d. $\mathcal{N}(0,1)$ offsets per n, fixed seed, the same `fit_line`):

$$\text{ratio}(n) = \frac{\text{median } s(\text{fit residuals})}{\text{median } s(\text{offsets})}, \qquad \text{inflation}(n) = 1/\text{ratio}(n). \tag{7}$$

Training targets become $\log(s \cdot \text{inflation}(n) + 0.02)$ for the scale and $r_k \cdot \text{inflation}(n)$ for every per-word residual (`configs: unfit_targets: true`). The model then learns pre-fit ("physical") wobble directly.

Parameters. The table depends on no data (`runs/unfit_ratio.json`):

n	3	4	5	6	7	8	9	10	11	12
inflation(n)	2.505	1.52	3.159	2.076	1.644	1.408	1.351	1.29	1.256	1.234

The table is non-monotone: it jumps where the model order rises at $n = 3$ and $n = 5$.

Evidence. Plan-level line C2ST fell from 0.638±0.006 (`v2_history`, fitted targets) to 0.539±0.016 (`v2_history_unfit`) (`runs/eval/ablation.md`, 5 seeds). On the validation split the old-table model does not beat v1 on line C2ST (§7.2). The independent review then showed that the table is wrong for single words (M2). It matches the median robust *scale* of a line, but it was also applied to every per-word residual target. For $n = 5$ the inflated targets have a per-word RMS of 2.07 times the physical σ, while their MAD is correct. The C2ST gain is therefore partly an artefact: the evaluation measures the same median-of-MAD statistic that the table was calibrated on. The coordinator reproduced the overshoot by simulation (Table 1).

4.5 Undoing the fit shrinkage, attempt 2: per-word leverage

Method. For ordinary least squares with design matrix V (centred powers of x, of degree 2 if $n \geq 5$, 1 if $n \geq 3$, else 0), the hat matrix is $H = V(V^\top V)^{-1}V^\top$. An i.i.d. offset of standard deviation σ then leaves residual i with standard deviation $\sigma\sqrt{1-h_i}$, where $h_i = H_{ii}$ is that word's leverage. The per-word correction is

$$\tilde{r}_i = r_i \cdot \min\Big(\frac{1}{\sqrt{1-h_i}},\, 4\Big) \tag{8}$$

(`unfit.word_inflation`). It uses the line's real word positions, and restores standard deviation σ at every n and every position. The cap of 4 guards words that nearly fix the fit. The line-scale target $\log(s \cdot \text{inflation}(n) + 0.02)$ keeps the table, which is correct for that statistic. Configuration: `configs/default_unfitw.yaml` (`unfit_words: true`); checkpoint `runs/default_unfitw/model.pt`.

Parameters. The leverages h_i are computed per line from that line's word centres. No data-fitted constant is involved. The MDN weights are retrained on the new targets: best epoch 152, stopped at 183 (`runs/default_unfitw/metrics.json`).

Evidence. Simulation (FACTS item 4): i.i.d. wobble with standard deviation 0.150 is placed at real train-line word positions and refitted with `dataset.fit_line`.

n	3	4	5	6	7	8	9	10	11	12
raw fit residual sd	.087	.107	.098	.109	.115	.121	.124	.126	.129	.132
leverage-corrected	.151	.149	.142	.147	.148	.150	.150	.149	.149	.149
old table	.219	.163	.311	.227	.190	.170	.168	.163	.162	.159

Table 1: Per-word standard deviation recovered from i.i.d. wobble with true sd $0.150\,U$. The leverage correction recovers 0.142–0.151 at every n. The old table overshoots by up to 2.1× (at $n = 5$).

At plan level the leverage model has line C2ST 0.521 ± 0.019 and page C2ST 0.581 ± 0.026. The residual-scale W_1 for short lines falls from 0.039 to 0.009 ($n = 5$) and from 0.028 to 0.011 ($n = 6$–7). The controlled 2-D FFT C2ST returns to v1's level: 0.566 ± 0.025 against 0.653 ± 0.024 for the old table (Table 8).

What it does not fix. The leverage correction assumes i.i.d. wobble. Real wobble is serially correlated, with residual lag-1 -0.38 on the test split against about -0.48 for i.i.d. generation (`runs/eval/ablation.md`). Real residual scale also grows with line length faster than fit shrinkage explains (review M2). The long-line bins still differ from real (W_1 0.041 at n = 8–10, 0.062 at $n \geq 11$).

4.6 De-noised curvature (extreme deconvolution)

Motivation: why generated lines split end to end. BengaliScribe writes the drawn word and line polygons of every generated page, so extracted lines can be matched to the lines that were actually drawn (`runs/compare_unfitw/split_diag.py`; an extracted piece belongs to the drawn line whose word polygon contains its centre). In the end-to-end run of the leverage model (`e2e_unfitw`, method v2, 5 writers × 2 seeds), 89 of 462 drawn lines (19%) were split into two or more extracted lines, and 127 extracted lines were single pieces. For the manual layout it was 6 of 355 (1.7%). At the 113 split points (`split_why.py`), the step in median ink row between the last word of one fragment and the first word of the next has median $1.2\,U$ (10th percentile 0.72, 90th 1.83). The line chainer joins neighbours only within `JOIN_DEV` $= 1.0\,U$; 74% of the steps exceed it, and for 86% the fragment-merge residual is finite but at least $0.6\,U$. The steps are not a glyph effect: the ($mid-$headline) offset of generated words has an interquartile range of $0.19\,U$, narrower than the real $0.30\,U$.

The steps come from lines that bow too much. Let the *sag* of a line be $c_2\,(L/2)^2$ in U, where L is the distance between its first and last word centres. At plan level on the test split (seed 0, lines with $n \geq 5$; `sag.py`), the share of lines with $|\text{sag}| > 0.5\,U$ is 0.108 for real lines, 0.170 for v1 and 0.175 for the leverage model (Table 3). The leverage model's sag sd is 0.358 and $0.509\,U$ on lines of 5–6 and 7–8 words, against 0.265 and $0.322\,U$ for real lines.

Interpretation: curvature noise. The curvature head is sampled before the line's word count is known (Eq. 2), so it learns one distribution of *fitted* c_2 pooled over all line lengths. On lines with 5–6 words, the fitted c_2 is mostly the trajectory fit's response to per-word wobble and anchor noise. The noise model below puts numbers on this: on the train split, the measured curvature sd falls from 0.00129 (n = 5–6) to 0.00084–0.00092 for longer lines, while the noise-corrected sd stays at 0.0007–0.0009 for every n (Table 2). True curvature looks like a habit of similar size at every line length. The noise of short lines, drawn on lines of any length, then becomes real bow.

Method. Write the fitted curvature of line i as the true value plus fit noise,

$$\hat{c}_{2,i} = c_{2,i} + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, v_i), \qquad v_i = \bar{s}^2_{n_i}\left[(V_i^\top V_i)^{-1}\right]_{22}, \tag{9}$$

where V_i is the line's centred quadratic design matrix (columns 1, $x - x_m$, $(x - x_m)^2$) and $\bar{s}^2_n$ is the per-word residual variance for lines with n words. This is the ordinary-least-squares variance of c_2; it ignores the Huber weights of the actual fit and treats the words as independent. A single line's own estimate $\text{RSS}/(n-3)$ has only $n-3$ degrees of freedom, so $\bar{s}^2_n$ is the mean of $\text{RSS}/(n-3)$ over all train lines with the same n, with $n \geq 11$ pooled into one group (`features.curv_noise_var`). Following extreme deconvolution (Bovy, Hogg and Roweis, 2011 [1]), the MDN curvature head with mixture parameters (π_k, μ_k, σ_k) then describes the *noise-free* curvature, and each measured target is scored under the mixture convolved with its own noise:

$$p(\hat{c}_{2,i} \mid H_i, c_{1,i}) = \sum_{k=1}^{K} \pi_k\, \mathcal{N}\big(\hat{c}_{2,i};\ \mu_k,\ \sigma_k^2 + v_i\big) \tag{10}$$

(`nn_utils.MDN.nll(noise_var)`, `model.head_nll`). At generation time the curvature is drawn from $\sum_k \pi_k \mathcal{N}(\mu_k, \sigma_k^2)$, without the noise term. The re-measurement of a generated line adds its own fit noise back, as it does for real lines. Configuration: `configs/default_unfitw_dc.yaml`, which is `default_unfitw` plus `deconv: [curv]`; checkpoint `runs/default_unfitw_dc/model.pt`. This is the default model of `bscribe_run.py` and `relayout.py`.

Learned and estimated quantities. The per-n pooled per-word variances $\bar{s}_n^2$ are estimated on the train split; the design factor $[(V_i^\top V_i)^{-1}]_{22}$ is computed per line from its word centres. Over the 4050 train curvature targets, the mean noise variance v_i is 38.4% of the variance of the measured curvature target. The MDN weights are retrained with the same seed and schedule. All heads other than curvature come out bit-identical to `default_unfitw`; only the curvature head's likelihood changed.

word count n	5–6	7–8	9–10	≥ 11
measured curvature sd	0.00129	0.00092	0.00084	0.00087
mean noise sd $\sqrt{\bar{v}}$	0.00092	0.00053	0.00053	0.00047
implied true sd	0.00091	0.00075	0.00066	0.00073

Table 2: Curvature noise decomposition on the train split (`runs/compare_unfitw/curvnoise2.py`; output printed, not saved). Implied true sd $= \sqrt{\mathrm{var}(\hat{c}_2) - \bar{v}}$. Curvature is in U^{-1}.

Evidence. Table 3 re-measures generated plans on the test split. The deconvolved model brings the share of strongly bowed lines and the sag spread to the real values, and its measured curvature sd is within about 0.0001 of real for $n \leq 8$. For $n \geq 9$ it is *below* real (0.00073 vs 0.00112 at n = 9–10, 0.00071 vs 0.00086 at $n \geq 11$); these bins hold only 76 and 34 test lines. The share of lines whose largest step between adjacent word anchors exceeds $1\,U$ falls from 0.169 to 0.138 (real 0.099). Over 5 seeds at plan level (`runs/compare_unfitw/drift.py`; output printed, not saved), the curvature W_1 to real is 0.0002 for the leverage model and 0.0001 for the deconvolved one, while slope (0.0045 vs 0.0044), drift (0.158 vs 0.155) and pitch W_1 (0.109 vs 0.114) barely move. The validation NLL of the curvature head, scored on the measured curvature, is -1.004 nats against -0.975 for `default_unfitw`. Both numbers are predictive densities of the same measured target, so they can be compared; the deconvolved model's NLL uses the convolved likelihood (10). End to end, the split rate of drawn v2 lines falls from 19% to 11% (§7.6). Plan-level C2ST does not separate the two models: line C2ST 0.535 ± 0.012 against 0.521 ± 0.019 on test and 0.536 ± 0.027 against 0.527 ± 0.030 on val (§7.2). The deconvolved model is the default because of the curvature statistics and the end-to-end splitting, not because of C2ST.

	$\lvert\text{sag}\rvert > 0.5\,U$	sag sd (U)		measured curvature sd				max. adj.
	share, $n \geq 5$	n = 5–6	7–8	n = 5–6	7–8	9–10	≥ 11	step $> 1\,U$
real (test)	0.108	0.265	0.322	0.00125	0.00083	0.00112	0.00086	0.099
v1	0.170	—	—	—	—	—	—	0.173
v2 leverage	0.175	0.358	0.509	0.00146	0.00113	0.00105	0.00113	0.169
v2 deconvolved	0.093	0.260	0.314	0.00117	0.00077	0.00073	0.00071	0.138

Table 3: Line bow at plan level, test split, seed 0, generated plans re-measured by `measure_page` (`runs/compare_unfitw/sag.py`, which takes the v2 checkpoint as its argument; output printed, not saved). Sag $= c_2(L/2)^2$. v2 leverage is `default_unfitw`; v2 deconvolved is `default_unfitw_dc`, the default. The last column is the share of lines whose largest step between adjacent word anchors exceeds $1\,U$. The bins n = 9–10 and $n \geq 11$ hold 76 and 34 lines. v1 is reported only for the two whole-population columns.

What it does not fix. The noise model is approximate: it ignores the Huber weights, assumes independent words although real wobble is serially correlated (§4.5), and pools $\bar{s}_n^2$ over all lines of a

length, so a line with unusually large wobble gets too small a v_i. The under-dispersion for $n \geq 9$ is consistent with the curvature head still not seeing the line length, but this was not tested, and the bins are small. Splitting is reduced, not removed (§7.6).

4.7 Huber-aware per-word correction (negative result, not adopted)

Motivation. The leverage factor $1/\sqrt{1-h_i}$ of §4.5 is exact for an ordinary-least-squares fit. The trajectory fit `dataset.fit_line` is a Huber IRLS fit, whose re-weighting shrinks the residuals of high-leverage words further. A new unit test (`tests/test_unfit.py`) shows the consequence: with i.i.d. Gaussian offsets, the leverage-corrected residuals of the first and last words of lines with $n = 4$–10 words stay about 10–25% below the drawn standard deviation. The leverage model therefore under-corrects the edge words.

Method. `unfit.word_inflation_huber` measures the factor on the actual fit. For each word count n and word position, the factor is sd(offset)/ sd(Huber-fit residual), estimated by passing 4000 i.i.d. $\mathcal{N}(0,1)$ offset vectors through `dataset.fit_line` with evenly spaced words. The cap of 4 is kept, lines longer than 30 words interpolate the $n = 30$ row over relative position, and the table is cached in `runs/unfit_word_huber.json`. Unlike the leverage factor, it depends on word position only, not on the line's actual word spacing. The model `runs/default_unfitwh_dc` (`configs/default_unfitwh_dc.yaml`: `unfit_words: huber`, `deconv: [curv]`) differs from the default only in this per-word factor; the line-scale target keeps the table.

n		Huber-aware (edge … edge)	leverage (edge … edge)
5	all words	4.0 (cap), 1.14, 1.46, 1.14, 4.0 (cap)	2.96, 1.26, 1.39, 1.26, 2.96
6	all words	2.91, 1.13, 1.20, 1.21, 1.13, 2.89	2.37, 1.20, 1.26, 1.26, 1.20, 2.37
8	edges / interior	2.11, 2.13 / 1.10–1.17	1.85, 1.85 / 1.14–1.18
10	edges / interior	1.76, 1.74 / 1.08–1.15	1.62, 1.62 / 1.11–1.18
12	edges / interior	1.55, 1.57 / 1.06–1.16	1.49, 1.49 / 1.09–1.18

Table 4: Per-word inflation factors for evenly spaced words. Huber-aware: `runs/unfit_word_huber.json`. Leverage: `unfit.word_inflation` evaluated on the same evenly spaced positions for this table. The Huber-aware factors are larger at the line ends and slightly smaller inside the line. The small left–right asymmetry of the Huber-aware values is simulation noise.

Evidence. The plan-level protocol of Table 8, 5 seeds, on test and on val (`runs/compare_unfitw/compare_5seed_wh_test.py` and `compare_5seed_wh_val.py`, outputs in the matching `.txt` files):

split	per-word factor	line C2ST	page C2ST	FFT C2ST (contr.)	resid. scale W_1 $n = 5/6$–$7/8$–$10/11+$
test	**leverage (default)**	0.535 ± 0.012	0.590 ± 0.024	0.550 ± 0.020	.007 / .011 / .042 / .061
	Huber-aware	0.544 ± 0.012	0.614 ± 0.011	0.583 ± 0.029	.009 / .014 / .043 / .068
val	**leverage (default)**	0.536 ± 0.027	0.593 ± 0.031	0.535 ± 0.044	.011 / .011 / .035 / .069
	Huber-aware	0.551 ± 0.022	0.619 ± 0.043	0.551 ± 0.049	.010 / .013 / .041 / .073

Table 5: Huber-aware against leverage per-word correction, both with the curvature deconvolution: `runs/default_unfitwh_dc` against the default `runs/default_unfitw_dc`. 5 seeds, mean ± sd (`runs/compare_unfitw/compare_5seed_wh_test.txt` and `compare_5seed_wh_val.txt`).

Measured: the Huber-aware model is worse on all three C2STs on both splits, six of six comparisons in the same direction. Each single difference is small, from +0.009 (test line C2ST) to +0.033 (test FFT C2ST), or about 0.3 to 2 seed-sd depending on the comparison and on which model's sd

is used; the evidence is the consistent direction, not any one gap. The residual-scale W_1 is higher in 7 of 8 bins (lower only at $n = 5$ on val). Both splits had already been used (§7.2), so this is a development comparison; since the candidate was rejected, it does not flatter the default's numbers.

Interpretation (not verified further): the Huber-aware factor is exact only for i.i.d. wobble. Real wobble is smoother than i.i.d.: the residual lag-1 correlation is -0.38 on real lines against about -0.48 for i.i.d. generation (review M2, §4.5). The edge residuals of real lines also carry line-end detection noise. Restoring the full i.i.d. edge variance, with factors up to the cap of 4 at $n = 5$, then probably overshoots at the line ends, and the leverage factor's smaller edge inflation happens to sit closer to what real lines need. Neither part of this explanation was tested on its own.

Decision and open fix. The default stays `runs/default_unfitw_dc`. The leverage factor is known to be inexact; it is kept because it scores better, not because it is right. The proper fix, not attempted, is to calibrate the per-word correction under a correlated-wobble model fitted to the real lag correlations, with the same Huber fit, rather than under i.i.d. offsets.

4.8 Word rotation along the trajectory

Motivation. BengaliScribe pastes words unrotated, so on a curved or sloped line each word's headline stays horizontal while the line moves. v1 measured word tilt but did not model it.

Method. Each word is rotated about its ink centre at the headline row (`write_page.rotate_word`), by

$$\theta_k = \beta \cdot \arctan\big(c_1 + 2c_2 d_k\big). \tag{11}$$

Parameters. $\beta = 0.524$, a Huber IRLS regression of measured headline tilt on the line tangent angle over 26 653 train words. On val, $\beta = 0.531$ over 5 237 words (`runs/default_unfitw/tilt.json`). The fitted intercept (-0.009 rad) is not applied.

Evidence. The review checked the pivot and the sign convention. No metric isolates rotation: the line features do not include word tilt.

4.9 Ink-extent placement

Motivation. The model's gaps are measured from ink edge to ink edge. BengaliScribe word images carry blank margins, so placing words by canvas width made every gap about $0.5\,U$ too wide.

Method. Gaps are applied between the ink extents of each word image, using columns darker than `INK_LEVEL` $= 0.35$ (`bscribe_layout.ink_extents`).

Parameters. `INK_LEVEL` was set by hand to match the real binarisation, where the Otsu cap of 170 keeps grey values darker than about 0.33. On generated pages Otsu gives about 0.39, so right ink edges come out $0.063\,U$ short and every measured gap is about $+0.067\,U$ too wide (review minor 2). This is not fixed.

Evidence. End-to-end, with the biased C2ST and the same model (`e2e_unfit` → `e2e_unfit_ink`): the v2 gap W_1 fell from 0.602 to 0.471, and v2 with writer context from 0.381 to 0.207 (Table 11).

4.10 Planner reset per page

Motivation. Training builds the page history afresh for every page. BengaliScribe, however, built one `PagePlanner` per document, and its history ran across 3–5 pages. The history count reached 56–72 lines, against at most 16 in training. History-conditioned slopes random-walked: one page mean was -0.158, reaching the -0.2 clamp, and 2 words were drawn past the page bottom (review C3).

Method. Each new page restarts from one fixed reference state: the real context page if one is given, else the state at the end of the first generated page (`bscribe_layout.layout_v2`, `snapshot`). All pages of a document therefore share one writer without drifting.

Evidence. After the fix, no words fall past the bottom. Page slope means stay near the writer's: for image_104 with context they are -0.053, -0.054, -0.074, -0.009 and -0.016, against -0.023 on the real page.

4.11 Margin guard

Motivation. Found while writing the first tests for `bscribe_layout` (§8). The page-break check looked only at the line centre: a new page started when center $+ 1.6 \cdot$ body $> H - m_b$. The line's slope, bow and the words' own ink extent were ignored. A sloped or bowed first line could rise about 1–1.4 U into the top margin, and a slope could carry ink below the bottom margin.

Method. In `bscribe_layout.layout_v2`, `ink_rows` gives the top and bottom ink rows of every generated word image (same `INK_LEVEL` as §4.9). `ink_span` returns the line's ink top and bottom along its current trajectory (slope and curvature, after the clearance step of §4.3), widened by `RESID_MARGIN` for the per-word wobble that is drawn afterwards.

- *Bottom margin.* If the line's ink would pass the bottom margin (or the old centre test fires), the line starts a new page and is *replanned from scratch*: its words go back to the queue, and the discarded draw never enters the page history. The new page therefore starts from the reference state of §4.10, not from a state that already contains the rejected line.
- *Top margin.* A page's first line is pushed down until its ink clears the top margin.

Parameters. `RESID_MARGIN` $= 0.3\,U$, set by hand to about 2 sd of the drawn wobble. Wobble beyond it, and small rotation effects, can still cross a margin.

Evidence. `tests/test_bscribe_layout.py::TestWordsStayInsideMargins` lays out synthetic word images over 2 seeds $\times$ (no context, context) and asserts that every word's ink lies within the margins $\pm 0.5\,U$. It passes on the new code and *fails on the old code*, where the highest ink row is at 178 px against a limit of 216 px. End to end, the guard changes no metric beyond noise (`e2e_final`, §7.6).

4.12 Balanced grouped C2ST for the end-to-end check

Motivation. In the end-to-end check every page is its own group and holds a single label. Grouped k-fold then changes the class balance between folds, the logistic intercept absorbs it, and pooling the out-of-fold scores pushes the AUC below 0.5. The old end-to-end real-vs-real "floor" was 0.34 (line) and 0.115 (page), which shows the protocol was broken (review C1).

Method. Groups are dealt into folds separately per label, and the reported AUC is the mean of the per-fold AUCs (`evaluate.grouped_kfold_auc(balanced=True)`; end-to-end only). The plan-level C2ST is unchanged, because there every group holds both labels.

Evidence. The new real-vs-real floor over 5 seeds is 0.503 (line) and 0.527 (page). In the `e2e_pagereset` run it is 0.534 and 0.508 (`runs/e2e_pagereset/results.md`). C2ST values from before and after this change are *not comparable*. Tables 11 and 12 are kept separate for this reason.

4.13 Latent writer-style model (not the default; unresolved)

Motivation. The history model represents a writer only through running means. An alternative is a writer-style vector z that is fixed for the whole page (`style: latent`). The line heads are conditioned on z plus a short previous-line term, and a small autoregressive MDN density over z draws it when no context is given.

Bug and partial fix. During training, z is a leave-one-out summary over almost the whole page. With writer context, z comes from half a page. This mismatch in the precision of z hurt context generation. A causal-prefix z (`runs/latent_unfit_fix`) improved latent+context: slope W_1 0.00977 $\rightarrow$ 0.00406, drift W_1 0.262 $\rightarrow$ 0.106, line C2ST 0.561 $\rightarrow$ 0.547. For comparison, history+context gives 0.00332, 0.110 and 0.540. The same fix *regresses* the latent model without context: slope W_1 0.00533 $\rightarrow$ 0.01283 and page C2ST 0.684 $\rightarrow$ 0.829 (`runs/eval/ablation.md` vs `runs/eval_latentfix/ablation.md`). The likely cause is that one fixed z is reused at every line position, while the precision of z in training depends on the position. This is unresolved, and the latent model is not used.

5 Learned and estimated parameters

Table 6 lists every constant that shapes the v2 layout, together with its value and where it comes from. Extraction thresholds that define *what is measured* are as in v1 and are not repeated.

name	value	how estimated	split
true wobble var. σ_t^2	$0.00906\,U^2$	$\mathrm{cov}(r_A, r_C)$ of the trajectory-fit residuals, lines with $\geq$ 5 headline words, 21 880 words, 1/99% winsorising	train
headline noise var. σ_A^2	$0.01176\,U^2$	$\mathrm{var}(r_A) - \sigma_t^2$	train
mid-row noise var. σ_C^2	$0.00631\,U^2$	$\mathrm{var}(r_C) - \sigma_t^2$	train
headline weight w	0.349	inverse-noise weight $\sigma_A^{-2}/(\sigma_A^{-2} + \sigma_C^{-2})$	train
combined noise var.	$0.00411\,U^2$	$(\sigma_A^{-2} + \sigma_C^{-2})^{-1}$	train
headline noise share	0.565 (plausible 0.38–0.66)	$\sigma_A^2/\,\mathrm{var}(r_A)$; range from review M3	train
combined noise share	0.31 (0.29–0.33)	$\sigma_{\mathrm{comb}}^2/(\sigma_t^2 + \sigma_{\mathrm{comb}}^2)$	train
wobble factor f_{wobble}	$\approx$ 0.83 (0.82–0.84)	$\sqrt{1 - \text{combined share}}$	train
clearance $d_{\min}$	$2.603\,U$	1st percentile of min. trajectory distance, 3 740 consecutive line pairs (val: 2.613)	train
clearance $\rho_{\min}$	0.797	1st percentile of min/mean distance (val: 0.788)	train
redraw limit	30	set by hand	—
inflation(n) (line scale)	2.505 … 1.234 ($n = 3..12$)	simulation of i.i.d. offsets through `fit_line`, 4000 lines per n, seed 0	none
per-word factor	$\min(1/\sqrt{1-h_i}, 4)$	OLS leverage of each word in the line's own fit; cap set by hand	none (per line)
per-word resid. var. $\bar{s}_n^2$ (curvature noise)	one value per n	mean of $\mathrm{RSS}/(n-3)$ over lines with n words; $n \geq 11$ pooled	train
curvature noise var. v_i	$\bar{s}_{n_i}^2[(V_i^\top V_i)^{-1}]_{22}$	OLS variance of c_2 from the line's own centred design	train ($\bar{s}_n^2$) / per line
curvature noise share	0.384	mean v_i over var. of the measured curvature target, 4050 lines	train
rotation gain β	0.524	Huber IRLS of headline tilt on tangent angle, 26 653 words (val: 0.531)	train
`INK_LEVEL`	0.35	set by hand to mimic the real binarisation (generated-page Otsu $\approx$ 0.39)	—
input/target μ, σ	—	standardisation statistics stored in the checkpoint	train
MDN weights θ (7 heads, $K = 4$, width 64)	—	maximum likelihood with teacher forcing; early stopping on val NLL (best epoch 152 of 183)	train (val for stopping)

Table 6: Learned and estimated parameters of the v2 default model (`runs/default_unfitw_dc`). Sources: `runs/anchor_noise.json`, `runs/line_clearance.json`, `runs/unfit_ratio.json`, `runs/default_unfitw/tilt.json`, `runs/default_unfitw/metrics.json`, `features.curv_noise_var` (curvature noise, §4.6), review M3 (ranges). The deconvolved model has the same rotation gain and the same best and stopping epochs as `default_unfitw`. No constant was fitted on test. Choosing *between* designs, however, used the test split (§9).

NLL (nats/target)	slope	curv	log res. scale	x_{start}	pitch	gap	resid
val	−2.363	−0.975	1.071	0.965	1.152	0.669	−0.606
test	−2.341	−0.954	1.073	0.844	1.176	0.679	−0.590
train rows	4942	4050	4050	4814	4258	25 546	26 878

Table 7: Per-head NLL of the leverage model `default_unfitw` (`runs/default_unfitw/metrics.json`). The default model `default_unfitw_dc` is bit-identical on every head except curvature; its validation curvature NLL, scored on the measured curvature under the convolved likelihood (10), is −1.004 (§4.6). The NLLs cannot be compared with those of models trained on different targets (fitted, table or leverage residuals), because the targets themselves differ.

6 Evaluation protocol

Plan-level. For each of the 193 test pages (unseen texts), every generator receives the page's real word widths and line breaks. Its plans are converted to raw coordinates and *re-measured* by `measure_page`, the code that measured the real lines. The run is repeated over 5 seeds, and we report mean ± sd across seeds. Each real page and its generated copy form one group in the grouped 5-fold C2ST, so every group holds both labels. Line C2ST features (lines with $n \geq 5$): mean and sd of the gaps, gap lag-1, slope, curvature, residual scale, drift, and the correlation of gap with width. Page features: mean and sd of line-mean gaps, mean and sd of slope, slope lag-1, mean curvature, mean residual scale, mean pitch. The classifier is logistic regression unless the table says MLP.

Nulls. The shuffled-real baseline (another test page's geometry) scores 0.584 at line level. It is *not* a real-vs-real floor: it pairs donor gaps with the target page's widths, which breaks the gap–width relation (0.018 vs real 0.130), and that relation is a C2ST feature (review M1). The comparable within-page null splits each test page's lines into two labels in the same group. It gives 0.496 ± 0.019 over 20 draws.

End-to-end. Five reference writers (test pages image_77, 104, 132, 147 and 910) × 2 seeds. BengaliScribe generates the words. The layout is `manual` (BengaliScribe's constants), `v2`, or `v2_ctx` (conditioned on the writer's own real geometry). The rendered pages go through the real detector and the same extraction, and are compared with the 193 real test pages. From `e2e_pagereset` onward the C2ST is balanced (§4.12). Recovered/placed is the ratio of extracted words to drawn words.

2-D FFT (controlled). A layout map is the filled word boxes at $8\,\text{px}/U$. It is windowed with a Hann window, and its 2-D power spectrum is summarised as radial PSD (32 bins), angular energy (18 bins) and the vertical line-pitch peak. In the *controlled* version, every generator runs on each real test page's own word widths. Real and generated pages are rendered with the same fixed box (anchor -0.25 to $+0.85\,U$) and cut to a $32 \times 18\,U$ window inside the real 10th-percentile text block. Single-feature confound AUCs are therefore 0.5 by construction.

1-D Fourier. Dense headline traces of real lines and of layout-realisable generated traces (rigid, rotated words) are compared by log-spectral distance in the band with wavelength $\geq 4\,U$.

7 Results

7.1 Plan-level, 5 seeds

method	line C2ST	page C2ST	2-D FFT C2ST (controlled)	resid. scale W_1, $n = 5$ / 6–7 / 8–10 / 11+
v1 (`scribe_layout/runs/default`)	0.566 ± 0.021	0.654 ± 0.043	0.567 ± 0.034	.011 / .025 / .055 / .074
v2, old table (`v2_history_unfit`)	0.539 ± 0.016	0.660 ± 0.004	0.653 ± 0.024	.039 / .028 / .029 / .058
v2, leverage (`default_unfitw`)	0.521 ± 0.019	0.581 ± 0.026	0.566 ± 0.025	.009 / .011 / .041 / .062
v2, deconvolved (`default_unfitw_dc`)	0.535 ± 0.012	0.590 ± 0.024	0.550 ± 0.020	.007 / .011 / .042 / .061
shuffled real (not a floor)	0.584 ± 0.016	0.605 ± 0.012	0.529 ± 0.050	.021 / .018 / .054 / .100
manual (BengaliScribe constants)	0.980	1.0	0.99	—
within-page real-vs-real null	0.496 ± 0.019	—	—	—

Table 8: Plan-level comparison on the test split (193 pages, 1495 lines). The v1 and old-table line and page C2ST values match `runs/eval/ablation.md` (`learned_full`, `v2_history_unfit`). The leverage row, the FFT column and the residual-scale bins come from `runs/compare_unfitw/compare_5seed.py` (5 seeds; output in `runs/compare_unfitw/compare_5seed.txt`); the deconvolved row comes from `runs/compare_unfitw/compare_5seed_dc.py` (output in `compare_5seed_dc.txt`). Bold: the current default. The FFT floor (real halves) is 0.483 for logistic regression and 0.522 for the MLP.

Measured. The leverage model has the lowest line and page C2ST among the learned models. Its line C2ST (0.521) is 0.025 above the within-page null (0.496), about 1.3 seed-sd. The deconvolved default model scores 0.535 ± 0.012 (line), 0.590 ± 0.024 (page) and 0.550 ± 0.020 (FFT). Every difference from the leverage model is smaller than the larger of the two seed-sds. The page C2ST of 0.58–0.59 still separates real from generated pages better than chance. No page-level null was computed for this protocol. The residual-scale W_1 is much lower than the old table's for $n = 5$–7, and still sizeable for long lines, for both leverage-family models.

W_1 to real (test, 5 seeds, mean)	slope	drift (U)	curvature	pitch (U)
v2 leverage (`default_unfitw`)	0.0045	0.158	0.0002	0.109
v2 deconvolved (`default_unfitw_dc`)	0.0044	0.155	0.0001	0.114

Table 9: Plan-level marginal W_1 of the two leverage-family models (`runs/compare_unfitw/drift.py`; output printed, not saved). Only the curvature W_1 moves. The script also prints the seed-to-seed sd: 0.053 and 0.058 for drift, 0.029 and 0.039 for pitch, far larger than the differences.

Interpretation, with caveats. The leverage correction removes the per-word overshoot without giving up the line-level gain. The curvature deconvolution leaves the plan-level C2ST and the slope, drift and pitch marginals where they were and changes the line bow (§4.6). Line C2ST 0.521–0.535 is *close* to the null, but these models were chosen after comparing alternatives on the same test split (§9). The review estimated the selection bias at about 1 sd. On the validation split (§7.2) the same two models score 0.527 and 0.536, within seed noise of their test values.

Earlier models (5 seeds, `runs/eval/ablation.md`). Line/page C2ST: `v2_history` (fitted targets) 0.638/0.752; `v2_latent` 0.650/0.749; `v2_latent_unfit` 0.565/0.684; `v2_history_unfit` with writer context 0.540/0.574. All learned models reproduce the dependence structure that the manual layout lacks (Fig. 1): gap lag-1 about 0.47–0.49 (real 0.475), gap vs. width (real 0.130) and slope persistence (real 0.884). The between-page ICC of mean gap is too high in all of them (0.45–0.53 against real 0.312). So is the clearance violation rate without rejection (0.09–0.15 against 0.014).

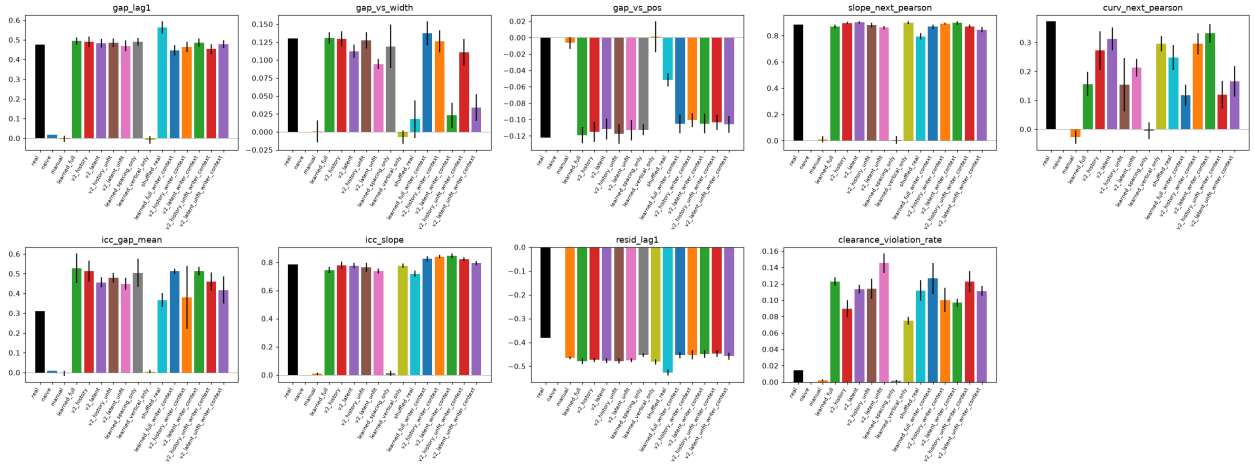


Figure 1: Dependence statistics on the test split, mean ± sd over 5 seeds (`runs/eval/`; generated before the leverage model existed, so it is not shown). Black: real. The manual layout (orange, third bar) has no gap persistence, no gap–width relation and no slope persistence. The last panel is the clearance-violation rate at plan level, without rejection sampling.

7.2 Validation-split check

Every design choice above was compared on the test split (review M4). To see how much of the test advantage survives on data that no comparison had used, the plan-level protocol was re-run unchanged on the validation split (4 texts, 144 pages), with the same 5 seeds, the same generators and the same controlled FFT set-up (`runs/compare_unfitw/compare_5seed_val.py`, output in `compare_5seed_val.txt`). No design choice or fitted constant was changed in response. The val split was not wholly unused before this check: it sets the early-stopping epoch of the MDN weights, and the clearance thresholds and the rotation gain were also computed on it as a cross-check (Table 6).

VAL split	line C2ST	page C2ST	FFT C2ST (contr.)	resid. scale W_1, $n = 5$/6–7/8–10/11+
v1	0.560 ± 0.024	0.631 ± 0.040	0.531 ± 0.033	.013 / .025 / .055 / .089
v2, old table (`default_unfit`)	0.566 ± 0.028	0.660 ± 0.022	0.599 ± 0.051	.049 / .030 / .024 / .050
v2, leverage (`default_unfitw`)	0.527 ± 0.030	0.575 ± 0.039	0.528 ± 0.024	.012 / .011 / .035 / .072
v2, deconvolved (`default_unfitw_dc`)	0.536 ± 0.027	0.593 ± 0.031	0.535 ± 0.044	.011 / .011 / .035 / .069
shuffled real (not a floor)	0.577 ± 0.023	0.597 ± 0.030	0.480 ± 0.034	.016 / .017 / .055 / .089

Table 10: Plan-level comparison on the *validation* split, 5 seeds, mean ± sd. Same protocol as Table 8. No within-page null was computed on val.

Measured. The two leverage-family models beat v1 on line C2ST (0.527 and 0.536 against 0.560) and on page C2ST (0.575 and 0.593 against 0.631), as on test. On the controlled FFT C2ST they tie with v1 (0.528, 0.535, 0.531). The old-table model, which beat v1 on test line C2ST (0.539 against 0.566), does not on val (0.566 against 0.560). Its FFT regression (0.599) and its short-line residual-scale overshoot (W_1 0.049 at $n = 5$) do replicate. The leverage model and the deconvolved model differ within seed noise on every C2ST, as on test.

Interpretation, with caveats. The improvement of the leverage-family models over v1 on line and page C2ST is not an artefact of selecting on test. The old table's test advantage over v1 on line C2ST (§4.4) was a selection effect, or at least did not generalise to other texts, so it should not be cited as an improvement. The val margins over v1 are about one seed-sd on line and page C2ST, and the split has only 4 texts, so the val numbers are noisy too. Having now been looked at, the val split can no longer serve as an untouched split for future choices.

7.3 FFT wobble-scale diagnostic

The old-table model improved line C2ST but *worsened* the controlled 2-D FFT C2ST, which suggested that its wobble was the wrong size. The diagnostic scaled the drawn residuals of `v2_history_unfit` and re-ran the controlled FFT C2ST (3 seeds, mean):

	v1	v2 fitted targets (`runs/default`)	v2 old table, drawn residual × 1.0	0.83	0.5	0.0
2-D FFT C2ST	0.612	0.628	0.683	0.658	0.578	0.601

Measured. The AUC falls as the drawn wobble is scaled down to $\times 0.5$, then rises again when the wobble is removed entirely.

Interpretation. The page spectrum detects the excess per-word wobble of the old table. Some wobble is still needed, since $\times 0$ is worse than $\times 0.5$. This motivated the leverage correction of §4.5, which lands at 0.566 ± 0.025 in the 5-seed run. These are 3-seed numbers from a separate script (`runs/compare_unfitw/fft_wobble_scale.py`, output in `fft_wobble_scale.txt`), and single-seed FFT AUCs move by up to ± 0.09. Compare them only with each other, not with Table 8.

7.4 2-D FFT layout-map experiment

Uncontrolled version: confounded. Detector-extracted real pages were compared with BengaliScribe-rendered pages. The spectral C2ST AUCs (logistic) were 0.958–1.0 for every method (`scribe_layout_fft2d/runs/results.md`). However, single non-layout features separate the sources just as well: text block height (single-feature AUC 0.008–0.017, i.e. 0.98–0.99 after flipping the direction, for manual, v1 and v2), median word-box width (0.90–0.93 after flipping, for v1, v2 and manual), and box height (about 0.81–0.82 for `v2` and `v2_old`). The uncontrolled AUCs measure text, page fill and glyph shape, not layout, and we draw no layout conclusion from them.

Controlled version: the main result. With text-matched word widths, a fixed box convention and a common window, all confound AUCs are 0.5 for every method. On a single seed (`results.md`), logistic C2ST was: manual 0.992, shuffled real 0.552, v1 0.552, `v2_history_unfit` 0.661, `v2_latent_unfit` 0.685. The real-halves floor was 0.483. The 5-seed values in Table 8 supersede these: v1 0.567 ± 0.034, old table 0.653 ± 0.024, leverage 0.566 ± 0.025, shuffled real 0.529 ± 0.050. Fig. 2 shows that the mean log-PSD of every learned model lies on top of the real one, while the manual layout has a strong spurious peak structure at low frequency (its regular line pitch).

What the spectrum is sensitive to. We ablated real pages, pooling the population. Shuffling the line pitch moves the mean log-PSD 6 times more (L2 0.080) than zeroing the slope (0.013) or removing the wobble (0.013). The page spectrum is therefore dominated by line-pitch regularity. The peak-prominence statistic is unreliable: it *rises* after periodicity is destroyed ($243 \rightarrow 273$).

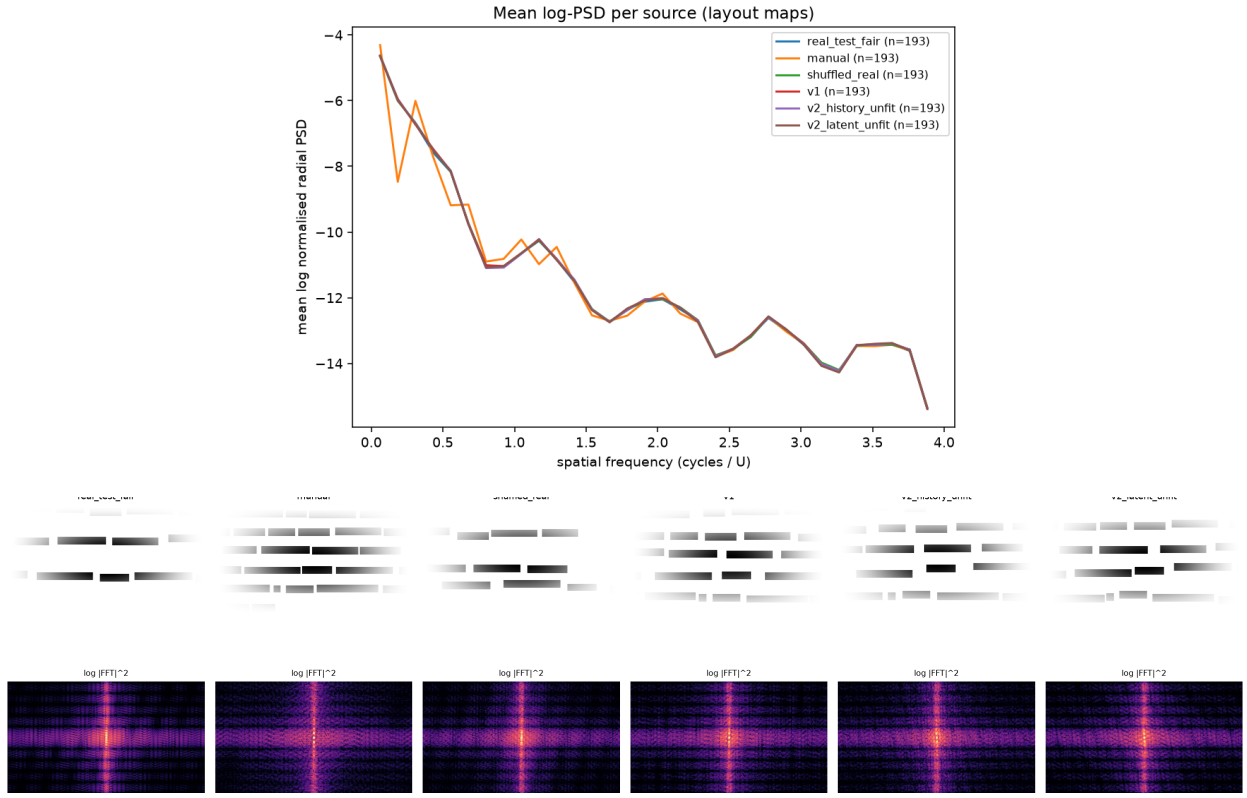


Figure 2: Controlled 2-D FFT experiment (`scribe_layout_fft2d/runs/figures/`). Top: mean log radial PSD per source, 193 pages each. All learned sources and shuffled real lie under the real curve, and manual (orange) departs from it at low frequency. Bottom: one page's layout map and log power spectrum per source. Learned models are included up to the old-table model, before the leverage model existed.

7.5 1-D Fourier experiment on headline traces

Method. Dense headline traces (`scribe_layout_fourier/`) were analysed by Lomb–Scargle PSD. Real wobble follows $\sim 1/f^{\alpha}$ with $\alpha = 1.06$ ($R^2 = 0.99$, train, 5099 lines). A spectral-synthesis generator draws traces with this spectrum.

Fair comparison. Words are rigid, so a layout can only realise a trace of rotated, rigid word segments. We compared layout-realisable traces in the band of wavelength $\geq 4\,U$ by log-spectral distance to real (`runs/fair_dense_psd.json`):

	manual	v1	Fourier	Fourier, continuous (not realisable)
distance to real	3.786	0.702	0.668	0.244

Interpretation. Once the comparison is restricted to traces a word-placement layout can actually produce, the Fourier generator and v1 are close (0.668 vs 0.702). The large advantage seen earlier holds only for continuous traces that rigid words cannot realise. An earlier claim that the Fourier model was "17× better" (3.98 vs 0.23 nats, `scribe_layout_fourier/runs/results.md`) was an artefact of comparing against that unrealisable continuous trace. A claimed "6× better x_{start}" came from including paragraph indents. At word level the Fourier model is worse on gaps (W_1 0.092 vs 0.038) and cannot reproduce the gap–width or gap–position relations. This matches the survey's prediction that spectra suit dense traces, not 4–8 gaps.

7.6 End-to-end

Tables 11 and 12 separate the runs that used the biased pooled C2ST from those that used the balanced one. Do not compare C2ST values across the two tables.

run (placement)	method	gap W_1	line C2ST	page C2ST	recovered
`e2e_unfit`	manual	0.685	0.893	0.998	1.004
(canvas width)	v1	0.582	0.699	0.942	0.797
	v2	0.602	0.750	0.901	0.969
	v1_ctx	0.526	0.653	0.971	0.714
	v2_ctx	0.381	0.595	0.901	0.910
`e2e_unfit_ink`	v2	0.471	0.667	0.840	0.952
(ink extents)	v2_ctx	0.207	0.532	0.918	0.866
real-vs-real "floor" (biased)		—	0.341	0.115	—

Table 11: **Biased-C2ST runs.** End-to-end runs with the pooled grouped C2ST, which is biased low (`runs/e2e_unfit/results.md`, `runs/e2e_unfit_ink/results.md`). Model: `default_unfit` (old table), with no planner reset. The real-vs-real values far below 0.5 show that the protocol was broken. The W_1 and recovered columns are not affected by the C2ST bias. They do depend on which lines survive extraction.

run (model)	method	gap W_1	slope W_1	drift W_1	pitch W_1	x_{start} W_1	line C2ST	page C2ST	recovered
`e2e_pagereset`	manual	0.685	0.035	0.965	1.839	0.948	0.905	0.999	1.004
(`default_unfit`)	v2	0.404	0.013	0.396	0.670	4.065	0.684	0.834	0.965
	v2_ctx	0.267	0.017	0.558	0.770	3.783	0.619	0.889	0.926
`e2e_unfitw`	manual	0.685	0.035	0.965	1.839	0.948	0.905	0.999	1.004
(`default_unfitw`)	v2	0.376	0.012	0.343	0.346	2.278	0.699	0.851	0.960
	v2_ctx	0.197	0.019	0.610	0.480	2.703	0.613	0.842	0.933
`e2e_unfitw_dc`	manual	0.685	0.035	0.965	1.839	0.948	0.905	0.999	1.004
(`default_unfitw_dc`)	v2	0.387	0.021†	0.645†	0.465	1.453	0.719	0.844	0.983
	v2_ctx	0.183	0.023†	0.695†	0.535	2.494	0.654	0.847	0.929
real-vs-real floor (balanced, 5 seeds)		—	—	—	—	—	0.503	0.527	—

Table 12: **Balanced-C2ST runs.** End-to-end runs with the balanced C2ST, planner reset per page and ink-extent placement. The three runs use the same protocol, writers and seeds; only the layout model differs. Sources: `runs/e2e_pagereset/results.md` (run floor 0.534 line, 0.508 page), `runs/e2e_unfitw/results.md` and `runs/e2e_unfitw_dc/results.md` (same run floor, 0.534 and 0.508). The manual rows are identical because the manual layout does not use the model. The line C2ST features omit x_{start} and pitch. †All W_1 values are computed on the lines that survive extraction. Slope and drift W_1 in particular are not comparable across runs with different split rates (see the survivor-bias analysis below). The `e2e_unfitw` slope and drift values were flattered by excluding split lines; the `e2e_pagereset` values very probably were too, but its split lines were not analysed.

Leverage model end to end (`e2e_unfitw`). Compared with `e2e_pagereset`, where only the model differs, the leverage model has lower gap W_1 (v2 0.404 → 0.376; v2_ctx 0.267 → 0.197), lower pitch W_1 (0.670 → 0.346; 0.770 → 0.480) and lower x_{start} W_1 (4.065 → 2.278; 3.783 → 2.703). Drift W_1 is better for v2 (0.396 → 0.343) and worse for v2_ctx (0.558 → 0.610). Line C2ST is essentially unchanged (v2 0.684 → 0.699; v2_ctx 0.619 → 0.613). Page C2ST moves in opposite directions (v2 0.834 → 0.851; v2_ctx 0.889 → 0.842) and stays far above the floor. The extractor found fewer lines on the generated pages (v2 499 → 456, v2_ctx 461 → 454) for about the same number of placed words. The pitch and x_{start} gains are *consistent with* less line fragmentation. Drawn-versus-extracted line counts were later measured for `e2e_unfitw` (below) but not for `e2e_pagereset`, so this comparison remains an interpretation. With 10 generated documents per method, changes in C2ST of a few hundredths are within noise.

Line splitting against the drawn lines. Matching extracted lines to the drawn lines (§4.6; `runs/compare_unfitw/split_diag.py`, method v2):

run (v2)	drawn lines	drawn lines split	single-piece extr. lines	extra fragments
`e2e_unfitw`	462	89 (19%)	127	113
`e2e_unfitw_dc`	469	52 (11%)	64	61
`e2e_final` (guard)	453	57 (12.6%)	80	69

The manual layout in `e2e_unfitw` has 6 of 355 drawn lines split (1.7%). The deconvolved model halves the single-piece lines and the extra fragments, and the v2 extraction keeps more lines (473 against 456) with a higher recovered/placed ratio (0.983 against 0.960).

Deconvolved default model end to end (`e2e_unfitw_dc`). Compared with `e2e_unfitw`, x_{start} W_1 falls (v2 2.278 → 1.453; v2_ctx 2.703 → 2.494), consistent with fewer mid-page fragments. Gap W_1 moves little (0.376 → 0.387; 0.197 → 0.183). Line C2ST rises slightly (v2 0.699 → 0.719; v2_ctx 0.613 → 0.654) and page C2ST is unchanged (0.844 and 0.847). Slope W_1 (0.012 → 0.021; 0.019 → 0.023), drift W_1 (0.343 → 0.645; 0.610 → 0.695) and pitch W_1 (0.346 → 0.465; 0.480 → 0.535) rise. At plan level, where nothing is split, the two models have the same slope, drift and pitch W_1 (Table 9).

Survivor bias in the end-to-end slope and drift. The drawn slope of every generated line can be read from its drawn line polygon (`runs/compare_unfitw/surv.py`, method v2; top edge of the polygon, with the bow as the range of the detrended top edge). In `e2e_unfitw`, the lines that the extraction split have median |slope| 0.046 (90th percentile 0.096) and median bow $0.66\,U$; the lines it kept whole have 0.026 (0.070) and $0.32\,U$. In `e2e_unfitw_dc`, split lines have median |slope| 0.052 and kept lines 0.025. Over *all* drawn lines of `e2e_unfitw_dc` the median |slope| is 0.027 (90th percentile 0.071), against 0.029 (0.086) measured on real test lines with $n \geq 3$.

Measured: split lines are the steep and bowed ones, and the drawn slopes as a whole are close to the real ones. *Interpretation:* the end-to-end slope and drift W_1 are computed only on the surviving lines. When 19% of the lines are split, the steep and bowed lines drop out of the metrics, the survivors look straighter and flatter than the generator's output, and the W_1 to real is flattered. When fewer lines are split, more of the steep lines enter the metrics. The rise in slope and drift W_1 from `e2e_unfitw` to `e2e_unfitw_dc` is therefore not evidence that the deconvolved model draws worse slopes, and the earlier end-to-end slope and drift numbers (Table 12) were flattered by excluding split lines. End-to-end slope and drift W_1 are not comparable across runs with different split rates. The drawn and real slopes come from different measurements (polygon top edge vs. trajectory fit), so their closeness is approximate. The rise in pitch W_1 is not explained by this analysis.

Final default end to end (`e2e_final`). Table 13 is the end-to-end result of the final default configuration: model `runs/default_unfitw_dc` with the margin guard of §4.11, same writers, seeds, balanced C2ST and real reference as Table 12. The layout code differs from `e2e_unfitw_dc` only by the margin guard.

method	gap W_1	slope W_1	drift W_1	pitch W_1	x_{start} W_1	line C2ST	page C2ST	recovered	pages	lines kept
manual	0.685	0.035	0.965	1.839	0.948	0.905	0.999	1.004	20	327
v2	0.425	0.021	0.672	0.390	1.239	0.719	0.850	0.981	30	440
v2_ctx	0.198	0.020	0.621	0.503	2.130	0.619	0.845	0.940	33	432
run real-vs-real floor	—	—	—	—	—	0.534	0.508	—	—	—

Table 13: **Final default end-to-end result** (`runs/e2e_final/results.md`): default model `default_unfitw_dc` plus the margin guard; balanced C2ST; 5 writers × 2 seeds; real reference 193 test pages. All W_1 values are on the lines that survive extraction (survivor bias, see above). Against the drawn lines (`split_diag.py`, v2): 57 of 453 drawn lines split (12.6%), 80 single-piece extracted lines, 69 extra fragments.

Measured: against `e2e_unfitw_dc`, line C2ST is 0.719 for v2 in both runs and moves 0.654 → 0.619 for v2_ctx; page C2ST moves 0.844 → 0.850 and 0.847 → 0.845. The split rate of drawn v2 lines is 12.6% (57 of 453) against 11.1% (52 of 469). The W_1 values move in both directions: gap 0.387 → 0.425 and 0.183 → 0.198, drift 0.645 → 0.672 and 0.695 → 0.621, pitch 0.465 → 0.390 and 0.535 → 0.503, x_{start} 1.453 → 1.239 and 2.494 → 2.130. The v2 method placed 2249 words against 2339. With 10 generated documents per method and no end-to-end seed-sd, we read the two runs as the same within noise. *Interpretation:* a replanned line very probably changes the random stream for everything drawn after it, so the two runs are best seen as different draws of nearly the same layout process, not as a paired comparison of the guard. The guard's effect is shown by the margin test (§4.11), not by these numbers.

Measured. With the balanced test, line C2ST ranges from 0.613 to 0.719 for the learned layouts and is 0.905 for manual. Page C2ST is 0.83–0.89 for the learned layouts, so real and generated pages remain easy to tell apart. The learned layouts reduce gap, slope, drift and pitch W_1 well below manual (on the surviving lines). x_{start} W_1 is worse than manual: 3.8–4.1 U with the old-table model, 2.3–2.7 U with the leverage model and 1.2–2.5 U with the deconvolved model (with and without the margin guard), against 0.95 U.

Interpretation, with caveats. The large x_{start} W_1 comes mostly from fragments of generated lines that the extractor splits apart: a fragment starts mid-page. Part of it is real margin spread. BengaliScribe also adds paragraph gaps and indents that the real hcr pages lack (review minor 7). The extraction splits generated lines heavily. In the pre-reset run `e2e_unfit_ink`, v2_ctx drew 507 lines; the extractor found 829 and kept 470. 39% of extracted lines were single pieces, against 12.9% on real pages. Against the drawn lines, 19% of v2 lines were split with the leverage model and 11% with the deconvolved model (12.6% in `e2e_final`), and the split lines are the steep or bowed ones. The metrics therefore see only the lines that survive extraction, and that subset is biased. Only about 50–60 generated lines with $n \geq 5$ per seed enter the line C2ST. Fig. 3 (middle) shows a v2_ctx page from the old-table model, where some words jump visibly off the line. This is the per-word overshoot of §4.4, which the leverage model is designed to remove.

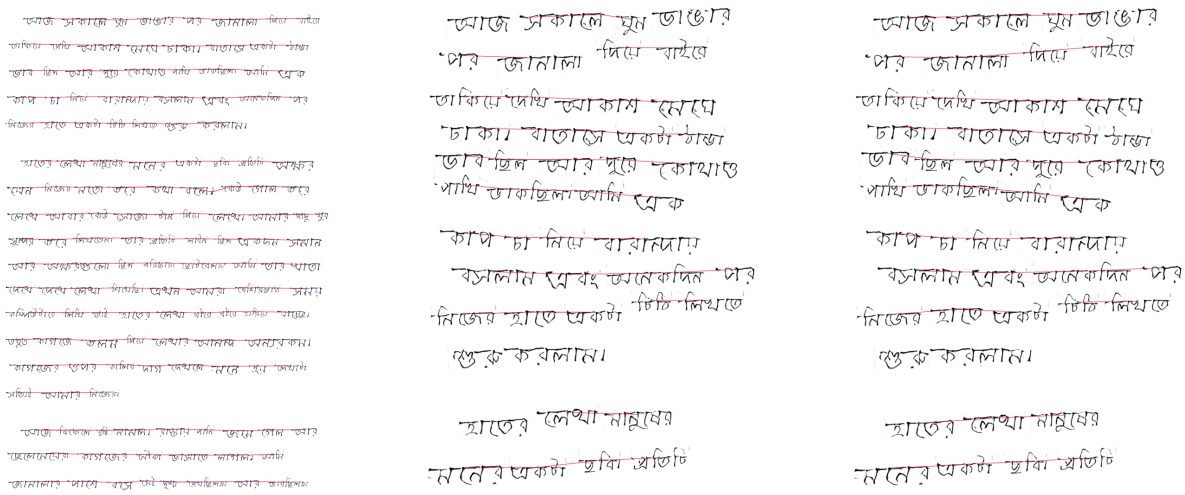


Figure 3: End-to-end pages for reference writer image_104 (`runs/e2e_pagereset/thumbs/`, `runs/e2e_unfitw/thumbs/`). Overlays are the lines re-extracted by the real detector and extraction. Left: manual layout, with straight, parallel lines and uniform spacing. Middle: v2 with writer context, model `default_unfit` (old table). Note the per-word vertical jumps, for example the raised last two words of line 2. Right: the same words, seed and context with the leverage model `default_unfitw`. The jump in line 2 is smaller, while the raised pair of words in the ninth line remains. One page cannot show a population effect. The figure illustrates the change and is not evidence for it.

7.7 Qualitative BengaliScribe comparisons

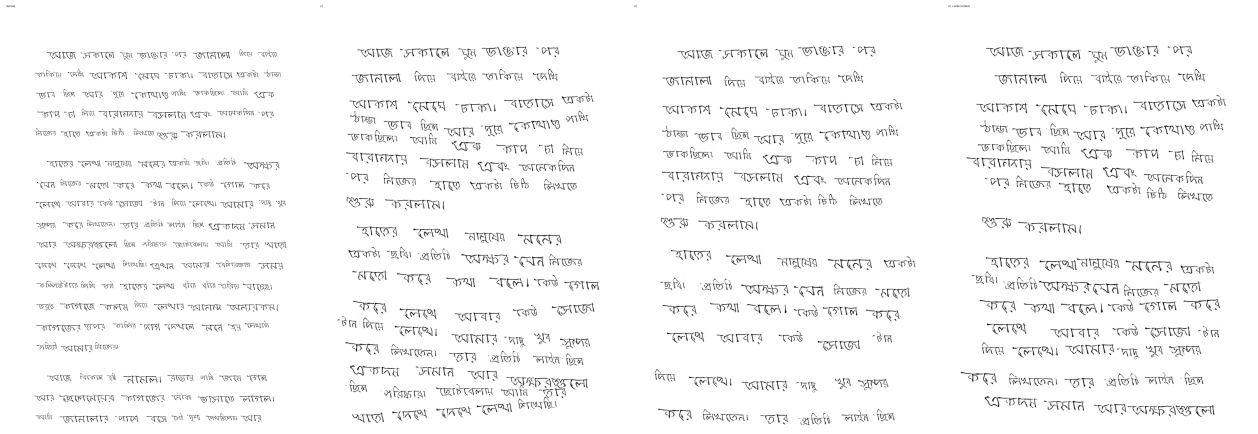


Figure 4: The same generated words under four layouts, top of page 1, reference writer image_132 (`runs/bscribe/compare_v1_v2.png`). From left: manual, v1, v2 (old-table model `default_unfit`), v2 with writer context. Rendered before the planner reset and before the leverage model.

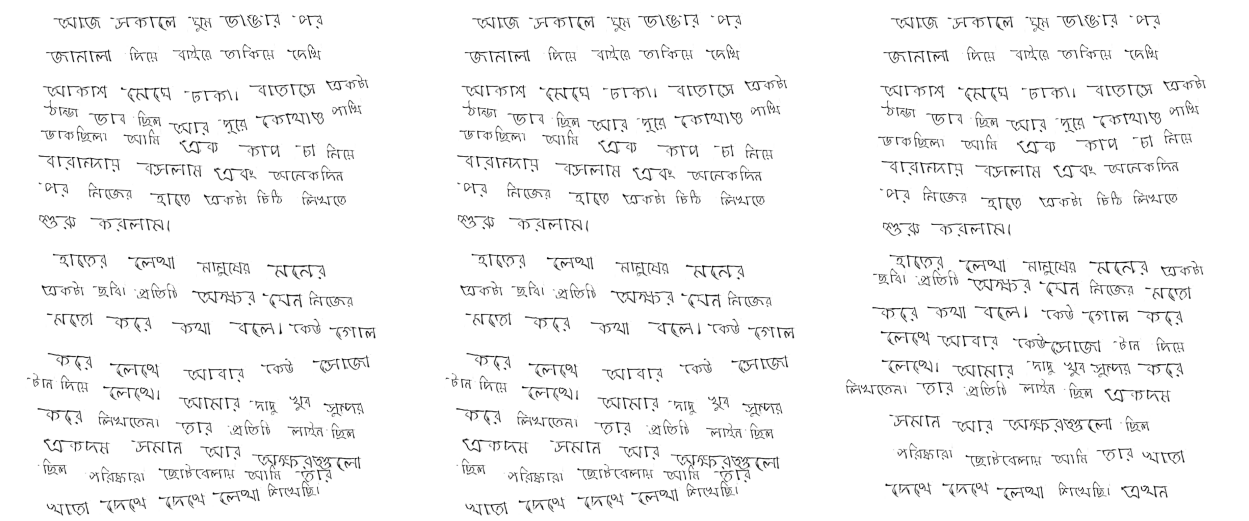


Figure 5: Re-layout of the same generated words (`relayout.py`; `runs/bscribe/compare_fixes.png`). Left: base (no de-noising, no clearance). Middle: wobble de-noised (×0.83). Right: de-noising plus line clearance. In the right panel the lines no longer converge or touch. The de-noising effect is subtle at this scale. Rendered with the model before the unfit change.

8 Corrections from independent review

An independent read-only review (`runs/review_opus.md`) reproduced each claim with a script (`c1`–`c19.py`). The table lists what it found and what changed. VERIFIED means reproduced by script. The last two rows are not from the review: they were found by the tests written in response to its "tests" finding.

id	what was wrong	how it was found	what changed
C1	End-to-end C2ST biased low by single-label groups; the reported v2_ctx line C2ST 0.532 did not hold (0.627 with per-fold AUC)	real-vs-real null 0.457±0.067 pooled vs 0.506±0.065 per-fold (20 draws); fold-seed range 0.41–0.61	balanced folds with per-fold mean AUC (§4.12); biased and balanced runs reported separately
C2	Extraction fragments generated lines; e2e metrics see only the surviving lines	line counts drawn, extracted and kept; re-render with zero wobble still fragmented	*reduced, not fixed*: slope drift (C3) and over-bowed lines (§4.6) were causes; 19% → 11% of drawn v2 lines split, 12.6% in `e2e_final` (§7.6)
C3	Planner history ran across pages; slopes random-walked to the clamp; words drawn past the page bottom	tracing `hist.count` (56–72) and per-page slope means	reset per page to a fixed reference state (§4.10)
M1	Shuffled real (0.584) presented as a real-vs-real floor	gap–width correlation 0.018 vs 0.130; within-page null	null reported as 0.496 ± 0.019; "below floor" wording dropped
M2	Unfit table applied to per-word targets: per-word RMS 2.07× physical at $n = 5$	simulation; resid lag-1 and length dependence of real wobble	per-word leverage correction (§4.5), on which the default model is built; serial correlation still not modelled

continued on the next page

id	what was wrong	how it was found	what changed
M3	"57% noise" rests on independence of estimator errors, which fails	tetrad test with 2 extra estimators; clip-level sweep	headline share reported as 38–66%; the combined share 0.29–0.33 used for generation
M4	All design choices compared on test; val never used	reading `evaluate.py`	*partly addressed*: an after-the-fact check on val (§7.2); designs still chosen on test (§9)
minor 2	`INK_LEVEL` 0.35 vs generated-page Otsu ≈ 0.39: gaps $+0.067\,U$	measuring ink edges	not fixed; documented
minor 3	Clearance check ran before the page-break decision	code reading	current code redraws the line shape under the new page's state when a line moves to a new page
minor 7	BengaliScribe adds paragraph gaps and indents that real pages lack	pitch after short lines on real pages (median 3.87 vs 4.44)	not changed; caveat on pitch and x_{start}
minor 10	`anchor_noise.json` stale against the cache (21 880 vs 21 794 words; w 0.349 vs 0.351); circular dependency	recomputation	negligible effect; reproduction order matters
tests	Tests that pass vacuously: noise-floor tolerance $\lvert\text{AUC} - 0.5\rvert < 0.4$; one-sample groups in the C2ST test; latent test on a fake model; no tests for `bscribe_layout`, `unfit`, `anchor_noise`, `line_clearance`	reading the tests against pathological outputs	*fixed or explained*: 70 tests, about 17 s (was 32, about 8 min). Noise-floor tolerance $0.4 \to 0.15$ (measured max. deviation about 0.08). Balanced grouped-k-fold tests with multi-row single-label groups (real-vs-real near 0.5; `balanced=False` biased low; shifted data > 0.8). New `test_unfit`, `test_bscribe_layout` (placement order across pages, margins, exact per-page history reset with and without context, clearance stats), `test_line_clearance`, `test_anchor_noise` (re-implements `main()`'s formulas on synthetic data, since `main` needs the real data). The latent test is kept on its fake model, with a docstring saying what it does not show
tests (new)	Margin bug: a sloped first line rose 1–1.4 U into the top margin; ink could pass the bottom margin	`TestWordsStay InsideMargins`, which fails on the old code	*fixed*: margin guard (§4.11)
tests (new)	Leverage factor under-corrects edge words by about 10–25% under the Huber fit	`test_unfit.py`	tried a Huber-aware factor; worse on every C2ST; *not adopted*, open (§4.7)

Table 14: Review findings and their status. Minor findings 1, 4, 5, 6, 8 and 9 are listed in `review_opus.md`. Minor 1 found the de-noising factor does not double-count. Minors 4–5 are training/sampling conditioning mismatches for short lines, suspected but not shown to matter. Minor 6 is unfiltered context lines. Minor 8 is writer overlap. Minor 9 is the inclusion of 5 context pages in the real reference.

The review's verdicts: plan-level line C2ST survives with caveats (correct null about 0.50; selected on test). The end-to-end line C2ST of 0.532 did not hold. The end-to-end gap W_1 survives as a measured number on the surviving lines, with a $+0.07\,U$ ink-edge bias. The noise share needs a range.

9 Limitations and threats to validity

- **Selection on test.** `evaluate.py` evaluates only on the test split. Fifteen methods were compared on test, and the unfit, de-noise, ink-extent, clearance and leverage choices were all iterated against test numbers. The fitted constants come from train (Table 6), so nothing leaked into the fits, but the test numbers are development-set numbers. The validation check (§7.2) softens this: with the designs frozen, the leverage-family models score 0.527 and 0.536 on val line C2ST against v1's 0.560, close to their test values, and beat v1 on page C2ST too. It also shows the risk was real: the old table's test advantage over v1 did not replicate on val. The val split informed no design choice, but it was looked at after the fact, it sets the early-stopping epoch, and it has only 4 texts. It is therefore a replication, not a clean held-out estimate, and it cannot serve as an untouched split for further choices. The Huber-aware correction (§4.7) was then judged on both test and val; it was rejected, so this does not flatter the default, but no untouched split remains.
- **No writer IDs.** There are 996 pages, about 37 copies of each of 27 texts, scanned in 57 batches, with no writer identifiers. The same people very probably wrote several texts, so the text-level split probably does not separate writers. Test numbers may therefore include writer overlap with train. This cannot be checked with the available data.
- **End-to-end line splitting.** The extractor splits generated lines far more often than real ones. Against the drawn lines, 11% of the default model's v2 lines are split (52 of 469), down from 19% (89 of 462) with the leverage model, and 12.6% (57 of 453) in the final run with the margin guard; the manual layout's rate is 1.7%. With writer context the rate is higher: 20% (92 of 464) in the final run, down from 26% (125 of 478) with the leverage model; some context writers slope close to the extractor's 0.15 slope limit (e.g. image_77, mean slope about -0.15 on generated pages). The split lines are the steep or bowed ones (§7.6); counts for all runs and variants are in `runs/compare_unfitw/split_counts.txt`. End-to-end metrics are computed on the surviving lines only, so slope and drift W_1 are biased towards straight lines and are not comparable across runs with different split rates, and x_{start} W_1 is inflated by fragments. The planner reset and the curvature deconvolution address two causes. The lines that still split are again the steeper ones (median |slope| 0.052), but why the chainer fails on them has not been analysed for this run.
- **The latent model regression is unresolved.** The causal-prefix fix helps context generation but hurts unconditional generation (page C2ST 0.684 $\rightarrow$ 0.829). The latent model is not used.
- **Noise-share range.** The headline-only noise share lies between 38% and 66%, depending on assumptions that the data partly reject. The generation-time factor (0.82–0.84) is robust to this range. Any statement about how much measured wobble is "real" is not.
- **Small end-to-end samples.** Five writers $\times$ 2 seeds give 20–35 generated pages per method and only about 50–60 lines with $n \geq 5$ per seed in the line C2ST. A single fold seed moved one old end-to-end line AUC between 0.41 and 0.61. End-to-end differences of a few hundredths are within noise.
- **Wobble model (open).** Per-word wobble is drawn i.i.d., but real wobble is smoother (residual lag-1 -0.38 against about -0.48 for i.i.d.) and grows with line length. Long lines remain off (W_1 0.061 at $n \geq 11$ for the default). The per-word correction is calibrated for i.i.d. offsets under least squares; it under-corrects edge words under the actual Huber fit, and the exact Huber-aware i.i.d. correction scores worse (§4.7), which suggests the i.i.d. assumption is the

larger error. Calibrating the correction under a correlated-wobble model fitted to the real lag correlations is the open fix; it has not been tried.

- **Margins (fixed).** Before the margin guard, a sloped or bowed first line could rise about 1–1.4 U into the top margin, and ink could pass the bottom margin. The guard (§4.11) fixes this, and a test that fails on the old code now covers it. It reserves only 0.3 U for the wobble, so an extreme word can still cross a margin by a little; the test allows 0.5 U.
- **Tests.** The 70 unit tests (§8) check code paths and invariants on synthetic data. They do not check that a trained model reproduces real statistics; that is what the plan-level and end-to-end evaluations are for. The latent mean-reversion test still uses a fake model.
- **Comparability across runs.** The leverage, deconvolved, Huber-aware and validation plan-level numbers come from comparison scripts, not from the standard `evaluate.py` run directory. Some of these scripts (`sag.py`, `drift.py`, `curvnoise2.py`, `split_diag.py`, `split_why.py`, `surv.py`) print their output without saving it. They use the same harness functions (`evaluate.generate`, `evaluate.c2st`), but the metrics files in `runs/eval` do not contain them.

10 Reproduction and files

From `/home/hpc4090/shyan/ocr`:

```
venv/bin/python -m scribe_layout_v2.anchor_noise # runs/anchor_noise.json (train)
venv/bin/python -m scribe_layout_v2.extract # re-extract with combined anchor
venv/bin/python -m scribe_layout_v2.line_clearance # runs/line_clearance.json
venv/bin/python -m scribe_layout_v2.unfit # runs/unfit_ratio.json
venv/bin/python -m scribe_layout_v2.train --config scribe_layout_v2/configs/
    default_unfitw.yaml
venv/bin/python -m scribe_layout_v2.train --config scribe_layout_v2/configs/
    default_unfitw_dc.yaml # default
venv/bin/python -m scribe_layout_v2.fit_tilt # runs/<name>/tilt.json
venv/bin/python -m scribe_layout_v2.evaluate # plan-level, 5 seeds -> runs/eval
venv/bin/python -m scribe_layout_v2.e2e_check --seeds 2 \
    --model scribe_layout_v2/runs/default_unfitw_dc/model.pt \
    --out scribe_layout_v2/runs/e2e_final --variants manual v2 v2_ctx # end-to-end (final)
venv/bin/python -m pytest -q scribe_layout_v2/tests # 70 tests, about 17 s
venv/bin/python -m scribe_layout_v2.bscribe_run --ref hcr/996/Images/image_132.png \
    --out scribe_layout_v2/runs/bscribe/v2_final --layout learned --context image_132
```

`anchor_noise` reads the extraction cache and `extract` reads `anchor_noise.json` (review minor 10), so the order matters for exact reproduction. `bscribe_run.py` and `relayout.py` default to `runs/default_unfitw_dc/model.pt`. The rejected Huber-aware model is trained with `--config scribe_layout_v2/configs/default_unfitwh_dc.yaml`; `python -m scribe_layout_v2.unfit` also writes its factor table `runs/unfit_word_huber.json`. The test suite (70 tests, about 17 s on this machine; 32 tests and about 8 min before) runs with the pytest command above.

The comparison scripts in `runs/compare_unfitw/` are run from the same directory as `PYTHONPATH=. venv/bin/python scribe_layout_v2/runs/compare_unfitw/<script>.py [args]`:

script (arguments)	output and use
`compare_5seed.py`	plan-level C2ST, FFT C2ST, residual-scale W_1, test, 5 seeds → `compare_5seed.txt` (Table 8)
`compare_5seed_dc.py`	the same for the deconvolved model → `compare_5seed_dc.txt` (Table 8)
`compare_5seed_val.py`	the same on the validation split → `compare_5seed_val.txt` (Table 10)
`compare_5seed_wh_test.py`, `compare_5seed_wh_val.py`	default vs Huber-aware per-word correction, test and val, 5 seeds → `compare_5seed_wh_test.txt`, `compare_5seed_wh_val.txt` (Table 5)
`fft_wobble_scale.py`	FFT C2ST vs drawn-wobble scale, 3 seeds → `fft_wobble_scale.txt` (§7.3)
`leverage_sim.py`	per-word wobble recovery → `leverage_sim.txt` (Table 1)
`curvnoise2.py`	curvature noise decomposition by n, train (Table 2); prints only
`sag.py` (*model.pt*)	line sag, curvature sd and adjacent steps, test seed 0 (Table 3); prints only
`drift.py`	plan-level slope, drift, curvature and pitch W_1, 5 seeds (Table 9); prints only
`split_diag.py` (*run_dir method*)	extracted vs drawn lines: split, single-piece, extra fragments (§7.6); prints only
`split_why.py` (*run_dir method*)	anchor steps and merge residuals at split points; (*mid*−headline) spread (§4.6); prints only
`surv.py` (*run_dir*)	drawn slope and bow of split vs kept v2 lines (§7.6); prints only

`extract.py`, `anchor_noise.py`	combined anchor; noise deconvolution
`unfit.py`	line-scale table, per-word leverage correction, Huber-aware factor (not used)
`features.curv_noise_var`	curvature noise variance v_i
`nn_utils.MDN.nll`, `model.head_nll`	convolved MDN likelihood (`deconv`)
`line_clearance.py`	real clearance thresholds
`fit_tilt.py`	rotation gain β
`features.py`, `model.py`, `train.py`, `api.py`	MDN chain, history and latent styles
`bscribe_layout.py`, `relayout.py`, `bscribe_run.py`	BengaliScribe integration (planner reset, margin guard)
`tests/`	70 unit tests
`evaluate.py`, `baselines.py`, `e2e_check.py`	plan-level and end-to-end evaluation
`scribe_layout_fft2d/`, `scribe_layout_fourier/`	2-D FFT and 1-D Fourier experiments

References

[1] J. Bovy, D. W. Hogg and S. T. Roweis (2011). Extreme deconvolution: inferring complete distribution functions from noisy, heterogeneous and incomplete observations. *Annals of Applied Statistics* 5(2B), 1657–1677.